

Real-time High Volume Data Replication

White Paper

Table of Contents

1	Meet HVR	3
2	HVR explored.....	7
2.1	Overview.....	7
2.2	Continuous Database Replication	9
2.3	Database Compare and Refresh	13
2.4	Managed File Transfer	15
2.5	HVR management and operations	16
2.6	Platform support	18
3	HVR usage scenarios	19
3.1	Real-time Business Intelligence	19
3.2	Application Integration.....	22
3.3	Distributed Computing	25
3.4	High Availability	27
3.5	Migrations	29
3.6	Heterogeneous Cloud Computing	30
4	Conclusion and Outlook	31
5	Further information	32

1 Meet HVR

Most companies and organisations rely ever more on intensive flows of information in order to carry out or sustain their business processes. The information needs to be available on the right time and at the right place. The related IT systems must therefore be capable of distributing large amounts of data to a multitude of computing platforms on geographically dispersed locations. Moreover, this must be achieved in a near real-time fashion.

This is exactly what HVR provides. HVR is a software package that can be deployed to configure and execute replications and synchronisations between various kinds of databases and other types of data repositories within distributed computing environments. With HVR, enterprises can manage very large and sophisticated data integration scenarios from a central point of control. HVR is specifically designed to scale out almost indefinitely and to make extremely effective use of network and computing resources, thus allowing for very small latencies.

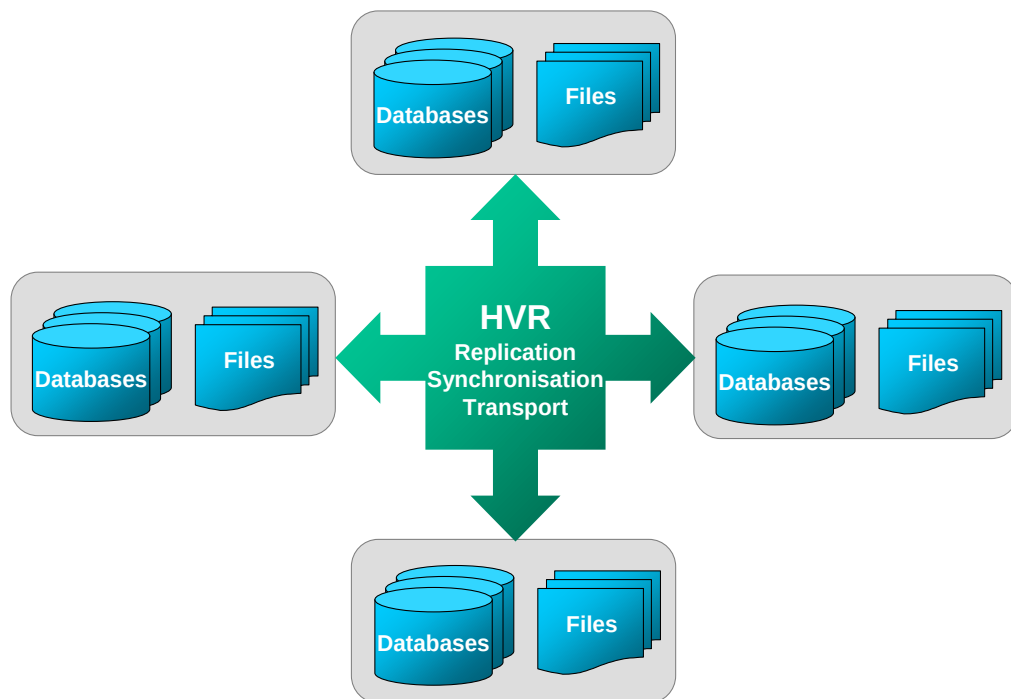


Figure 1 HVR core capabilities

HVR: the Company

HVR Software bv is a privately-held software company headquartered in Amsterdam, The Netherlands. The company was founded in 1995. Since 2000 it has been dedicated to the development and marketing of its software product HVR, which stands for “High Volume Replicator”. HVR was originally conceived to solve the need for replication of large databases over low speed networks within the logistics industry, and for high availability within the telecommunications industry. Since then, HVR has developed into the powerful tool that nowadays supports a multitude of sophisticated data integration, synchronisation and replication scenarios. Typical cases may involve relational databases of various types and vendors, as well as other types of data repositories. Customers now include businesses in many sectors, such as aviation, financial services, maritime and utilities.

Product Highlights

HVR offers a number of powerful features that can be grouped into three major functionalities. These are provided within a common software framework and can be managed from a single integrated management console.

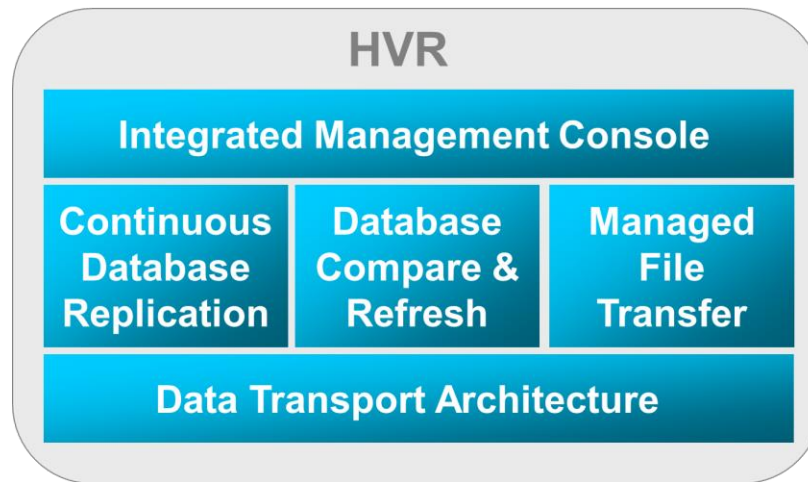


Figure 2 HVR framework

- **Continuous Database Replication**

With HVR, continuous replications can be set up and executed between databases within large distributed computing environments. The database replications are achieved through a mechanism called Change Data Capture. Changes applied to the source database are detected by HVR in real-time and transmitted over the network to be copied to one or more target databases in order to keep them identical.

The replication schemes can be as complex and large as required. A multitude of databases may be involved in a broad variety of topologies. The schemes can be configured, initiated and monitored from a single point of control within the enterprise and through a highly intuitive Graphical User Interface.

Replications may be applied to databases as a whole, but can also be done selectively on specific tables within a database or even rows within a table. Replications can be configured between instances of different DBMS products, such as Oracle Database, Ingres, IBM DB2 and Microsoft SQL Server.

HVR's software algorithms are optimized to make the replication processes as fast and efficient as possible. For instance, the change data is highly compressed when sending it over the network and the software always directly uses the native DBMS interfaces.

- **Database Compare and Refresh**

HVR can also be deployed to compare the contents of different databases. In case there are differences, HVR is able to equalize (refresh) the contents, thus bringing the databases in sync. This can for instance be required as a prerequisite for enabling Continuous Database Replication, or to recover a database taking part in a replication after it has crashed.

As with Continuous Database Replication, Database Compare and Refresh can be applied to heterogeneous DBMS instances and operates fast and efficiently.

- **Managed File Transfer**

Last but not least, HVR offers the capability to schedule, execute and monitor complex chains of file transfers. Files can be exchanged between multiple platforms, including Unix/Linux, Windows, Microsoft Sharepoint and FTP. Extensive and flexible rules can be applied for selecting, routing and renaming files.

Though functionally distinct, all the above functionalities are delivered within the same integrated software package and a common architecture. As a consequence, they enjoy the same benefits in terms of scalability, high performance, efficiency, and the intuitive graphical user interface of the control panel.

Business Benefits

HVR's unique set of capabilities make it a very versatile middleware tool that can be applied as a solution in a number of widely divergent usage scenarios.

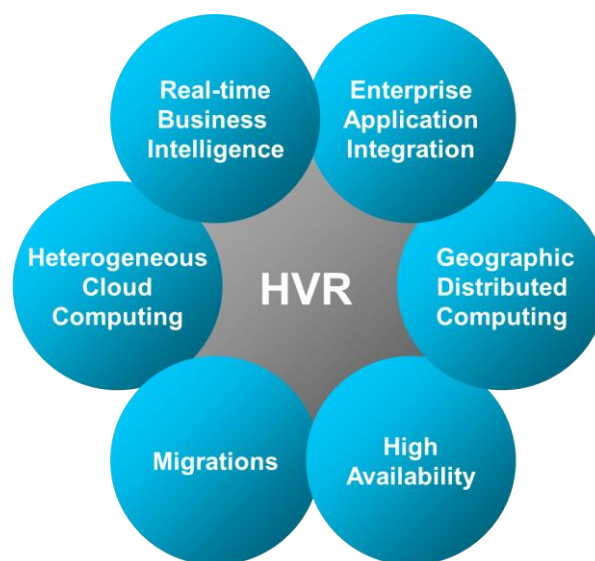


Figure 3 HVR usage scenarios

- **Real-time Business Intelligence**

Business Intelligence (BI) refers to the process in which data is extracted from the production applications and data stores in support of the primary business operations and subsequently combined and manipulated to produce information that can be used to support decision-making. Traditionally, the emphasis has been on tactical and strategic decision-making processes which are less time-sensitive. However, within today's competitive environment the need increases for up-to-date information in support of much more operational decision-making processes. This "Operational Business Intelligence" sets much higher requirements on the speed with which data can be extracted from various kinds of primary data sources and consolidated into a data store formatted for reporting purposes. HVR is able to fulfil these requirements through its specific real-time data integration capabilities.

- **Enterprise Application Integration**

For enterprises with their extensive software application portfolios it is a major challenge to make their applications work together to produce a unified set of functionalities tailored to their business needs. The approaches and associated software tools adopted to achieve this objective are either based on the principle that the applications work on a common data set (Data-oriented Application Integration), or on the principle that the applications directly interact with each other

through the exchange of messages or the invocation of procedures (Transaction-oriented Application Integration). Because of its unique capabilities regarding the continuous replication of databases and managed file transfers within heterogeneous environments, HVR qualifies as the middleware tool of choice to implement Data-oriented Application Integration.

- **Geographical Distributed Computing**

In many cases organisations need to spread their computing systems over a number of locations. Globally operating companies usually run their IT services from regional data centres to ensure local availability and performance. Organisations that operate on a national level may find it useful to retain data and computing facilities in their local branch offices, especially if the local availability of business applications is very business-critical and network performance is relatively expensive or unreliable. HVR can be deployed to selectively reproduce large databases and files between geographically dispersed sites and does this extremely fast (near real-time) and very efficient (minimal use of network capacity).

- **High Availability**

For business-critical operations it is often necessary to have multiple copies of the data and the databases available. This allows for system fail-over in case of incidents, data centre fail-over in case of disasters, and avoidance of down-time during system maintenance. It also enables load-balancing of concurrent systems to avoid degradation of performance. HVR can be used to create multiple database and file copies. Because of HVR's fast and efficient replication and transport mechanisms, data loss will be zero or very small in case of a fail-over.

- **Migrations**

In every large IT environment migrations are constantly being carried out. They may range from software upgrades and server installations to large scale projects implementing new platforms or business applications. Such migrations pose challenges that usually are a combination of availability and integration issues. Availability issues arise from the requirement to reduce or eliminate downtime caused by upgrades or switch-overs from old to new systems. Integration issues occur when the legacy systems have to interoperate with the new systems in the course of a migration. The combined benefits that HVR provides for Application Integration and High Availability scenarios therefore also make it a very adequate solution for migrations.

- **Heterogeneous Cloud Computing**

Cloud computing services offer major advantages in terms of cost flexibility, scalability, availability and deployment speed. Many companies are therefore migrating at least part of their IT environment to the cloud. As a consequence, their IT landscape develops into a combination of cloud hosted and internally hosted applications. They are faced with the challenge of exchanging, replicating and synchronizing the data between these applications for all of the purposes already discussed above in which HVR can be applied. HVR can be deployed on any Infrastructure-as-a-Service (IaaS) platform delivering virtual servers with Operating Systems that are supported by HVR, such as Linux and Windows instances within Amazon EC2. In addition HVR has been specifically adapted to interoperate with the databases that are included in the Salesforce and Microsoft Azure cloud offerings.

Outline

This white paper takes a closer look to HVR and the solutions it can provide to IT and business challenges. Chapter 2 provides a more in-depth description of the HVR software suite, both from a functional and from a technical point of view. Chapter 3 discusses the typical scenarios in which HVR can be deployed as a most adequate solution. Chapter 4 concludes with a summary of the main findings of this whitepaper and a brief outlook to future developments.

2 HVR explored

2.1 Overview

Architecture

Each HVR task, be it a Continuous Database Replication, a Database Compare and Refresh or a Managed File Transfer operation, is implemented within a common software framework using shared code components. HVR tasks operate on database and file stores that are generically referred to as “locations”.

The HVR architecture relies on collaborating software agents residing at each of the “locations”. The HVR agents are runtime processes executing HVR code on each server where a database is located or from which a file store is accessed. The HVR agents at the source locations take care of data capture from their local database or file store and sending the captured data to the agents at the target locations. The agents at the target locations integrate the data they receive into their local database or file store. The data exchange between agents uses proprietary HVR protocols and algorithms.

One server running an HVR agent has been assigned a central role and is called the HVR hub. The hub serves as the central point from which all HVR agents are controlled and HVR tasks are scheduled, monitored and logged. The HVR hub also interfaces with the HVR Graphical User Interface (GUI), where HVR tasks can be configured and loaded to the hub for execution. The hub agent requires a database to store all the HVR task parameters, such as names of the replicated databases and the actions to be performed. In contrast, all the other (“remote”) HVR agents are only executables without any associated database or log files.

The HVR hub also serves as the central queuing node for the data exchanges between source and target locations. Since it only requires restricted resources, the HVR hub is often deployed on one of the source or target machines, thus avoiding investments in dedicated hardware.

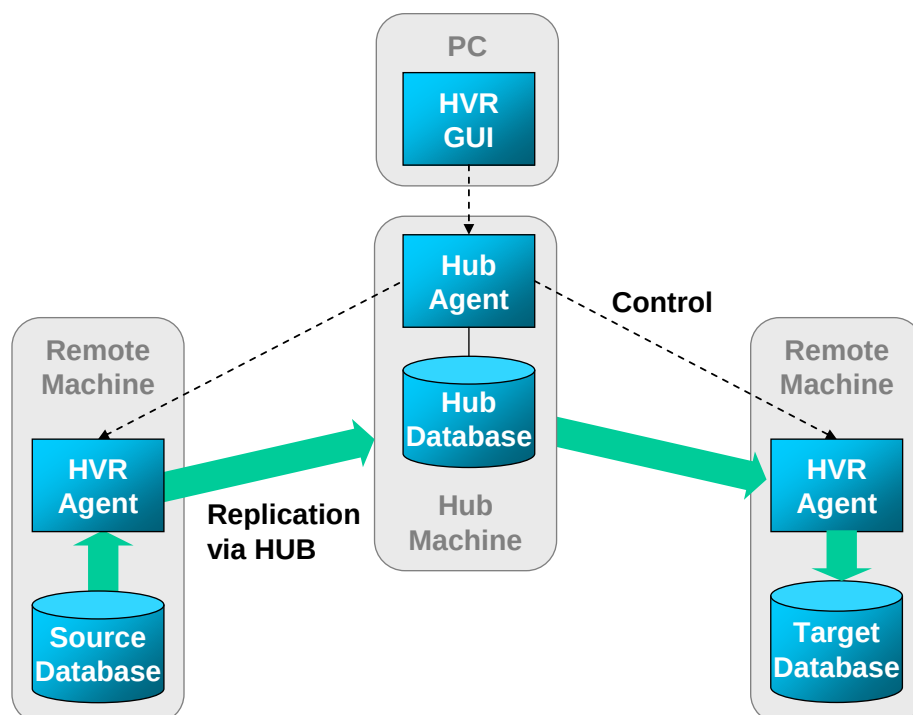


Figure 4 HVR architecture

HVR has a track record for operating in large distributed and heterogeneous environments thanks to its specific design criteria:

- *High performance and efficiency*
The HVR agents interface to the local databases or file systems through the native interfaces, avoiding any overhead due to compatibility layers, such as JDBC or middleware. The protocols and algorithms that the HVR agents use to communicate over the network are optimized for high speed and low bandwidth usage. This is achieved through powerful compression and smart data packaging techniques.
- *Scalability*
Most of the HVR processing is done by the local HVR agents capturing or integrating data. Adding additional locations to the topology also implies adding additional HVR agents. HVR can therefore scale to thousands of locations.
- *Availability and continuity*
If multiple locations are selected as the source or target of an HVR task, and one of the locations becomes unavailable in the process, HVR will still complete the task for the remaining locations. This is achieved by creating independent jobs handling the communication with each of the locations, especially on the hub. It should be noted that the hub itself may be a single point of failure, so this should preferably be installed on a highly available server. Moreover, it is possible to include a standby hub in the setup to which the HVR operation can be switched over when the primary hub fails. In this case, all HVR tasks can be continued without any disruptions or data losses.
- *Robustness and error recovery*
HVR has a range of options and mechanisms to detect and resolve errors that may occur, such as database collisions due to bi-directional replication, database errors, application errors and operator mistakes.

2.2 Continuous Database Replication

Through the Continuous Database Replication functionality of HVR, changes applied to one or more designated source database can be selectively replicated to one or more target databases. The databases may be of varying types or located on distributed sites. HVR is optimized for high speed operation, efficient use of network and system resources, and large data volumes.

The implementation of HVR Continuous Database Replication makes use of its architecture with collaborating HVR agents that reside on the database servers involved in the replication process and that directly connect to their local databases. The HVR agent on a source database server continually inspects the source database for changes, extracts the relevant changes and sends these over the network to the HVR agents on the target database servers. The receiving agents apply the changes to the local database as SQL transactions via the native interface of the database server. The consistency and integrity of the replicated data is preserved using the following principles:

- Changes are acknowledged after they have arrived and have been applied to the target database,
- Changes are not replicated until they are committed,
- Changes are re-played on the target database in the same order that they occurred on the source database.

This paragraph describes the primary technical features and components of Continuous Database Replication and explains how these contribute to HVR's unique capabilities.

Change Data Capture

Change Data Capture is the mechanism through which changes to a database can be detected and extracted. HVR uses Change Data Capture to retrieve changes to the source database for subsequent conveyance to the target database(s). Usually *Log-based Capture* is applied. This exploits the log files in which the database server records all transactions that are applied to its databases, such as the redo logs in an Oracle Database server or the transaction logs in an Ingres server. HVR normally directly inspects the on-line log files at the same time as they are filled with change records. However, if HVR falls a long way behind for whatever reason, it will resort to reading from the archive log files until it catches up again to the current point in the on-line log files. Log-based capture is completely unobtrusive to the database server as it only needs read-permission to the database log files. It does not directly access the database itself (i.e. the database server or the database files), and therefore no database locks or any other overheads will occur.

Alternatively, HVR can perform *Trigger-based Capture*. This makes use of a mechanism supported by most database servers to define "database triggers". A database trigger is a procedure that is automatically executed when a certain type of transaction, e.g. on a particular table, is applied. HVR uses triggers to record changes to tables within the source database into so called HVR capture tables. Unlike Log-based Capture, Trigger-based Capture does involve interference with the database and therefore additional overhead and latency. It does however also allow for specific customizations.

Network Transport

The changes that are captured from the source databases need to be conveyed to the target databases, which may be situated in sites connected via slow networks. For this purpose HVR features a built-in transport mechanism allowing for very efficient use of the available system and network resources.

The transport of change data is achieved over a direct socket connection between the HVR agents on the database servers

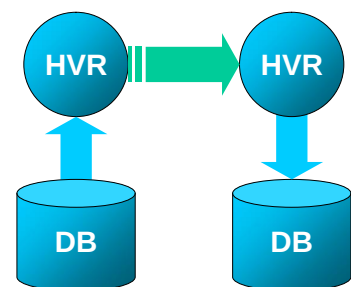


Figure 5 Network transport

involved in the replication process. This “symmetrical” streaming approach with collaborating HVR agents on both ends of each transport connection enables the following specific features and benefits:

- *Data compression*
HVR comes with a powerful tailor-made compression algorithm that exploits specific information on the transported data, such as the column data types and table widths involved. The resulting compression ratios vary between 85% and 99% , hence making very efficient use of the available bandwidth.
- *Data packaging*
Prior to transmitting the data, HVR combines whole queues of changes into only a few network packets, thereby minimizing the overhead of network protocols and performance reductions due to network round-trip latencies.
- *Data encryption*
Data encryption to protect sensitive data can optionally be applied by using Secure Socket Layer (SSL).
- *Bandwidth management*
HVR can be configured to only use a defined fraction of the maximum bandwidth on a network link, so that sufficient capacity remains available for other types of network traffic.
- *Coalescing of changes*
Multiple changes that are applied to the same row in the replicated database within a short time can be detected by HVR and coalesced before transmission. Coalescing means that the changes are replaced by a single change having the same result when it is applied to the target database. For instance, an insert and two updates to the same row can be merged into a single insert. Applying coalescing of changes reduces the number of changes to be transported and applied, which further boosts replication speed.
- *Minimization of replication hops*
The number of replication hops that occur when changes are buffered to disk between the captured database and the target databases should be kept as low as possible, since they cause disk I/O adding to overhead. HVR only needs to queue data changes to disk once, in order to keep the process running if one target database is offline, and the data that is written to disk is highly compressed.

Replication Topologies

HVR can both be used for *one-directional* and for *bi-directional* replication between two databases. In one-directional mode transactions can only be applied to a master database and the resulting changes are replicated to a slave database. Hence, the master database acts as the replication source and the slave databases as the replication target. Alternatively, in bi-directional mode transactions can be made to both databases and the resulting changes can travel in both directions between these databases. Either database acts both as the source and as the slave in the replication process.

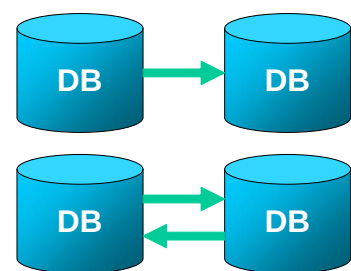


Figure 6 Replication directions

HVR is not only applicable for 1 to 1 replications, but it can be used in any topology, including 1 to N, N to 1, up to very complex meshed topologies with a large number of databases that each may have multiple incoming and outgoing replication streams. In all these topologies HVR routes the changes through the database server that has been designated as the HVR hub. The hub therefore serves as a central queuing node. Changes originating from a source database are initially sent to the HVR hub where they are stored in a queue file. From there on the change is sent to the target databases.

Especially in large topologies with many data flows, this central queuing mechanism greatly simplifies the configuration and management of the replication scheme and makes it more robust and scalable. Much less connections are required and it is easy to add or remove a database without disrupting the rest of the scheme.

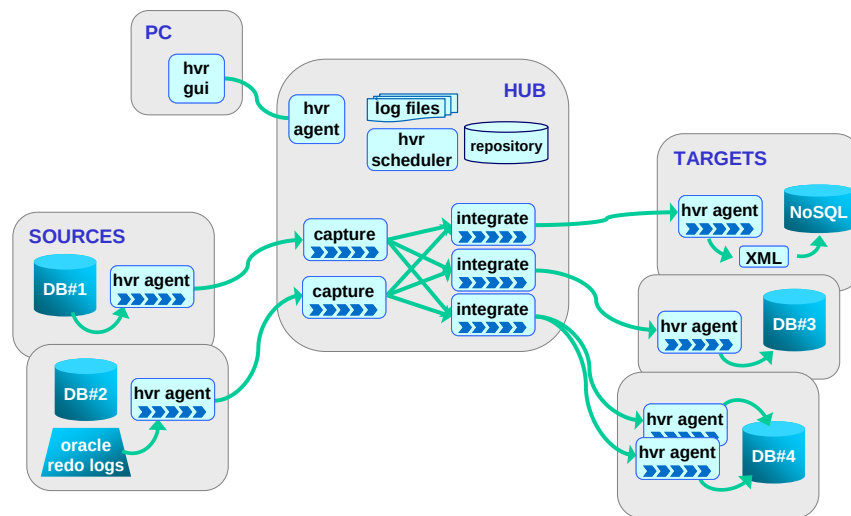


Figure 7 HVR change data flow

Collision Handling

Collisions can occur during database replication when a change that is applied locally to a database is overwritten by an earlier change arriving with some delay from a replicated source database. Undetected collisions can lead to inconsistencies within and between the replicated databases. HVR has an efficient collision detection and resolving mechanism using the timestamps of the changes involved. It can be configured to operate selectively on the databases and tables where collisions may occur, such as in a bi-directional replication process. If the replicated table contains timestamp columns these can be used without the need to maintain a separate timestamp table, reducing overhead.

Heterogeneous Replication

One of the major features that distinguishes HVR from many other database replication tools is that it is not bound to a single DBMS product. HVR can just as easily replicate data between an Oracle database and an Ingres database as between two different versions of Microsoft SQL databases.

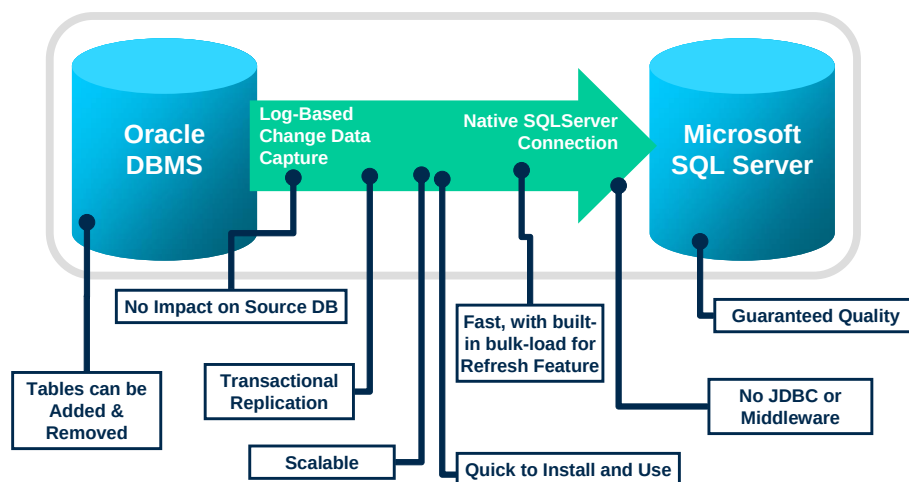


Figure 8 Example: HVR replication from Oracle Database to Microsoft SQL Server

HVR's performance and functional features remain unchanged in mixed DBMS environments, including bi-directional replication and collision handling. No additional tools or interfaces are required, as the local HVR agents directly interface with the local DBMS instance and take care of any necessary data type conversions.

Moreover, HVR provides the option to selectively replicate data from the source to the target databases, map between different table or column names and convert data values during the replication. The database schemas of the source and target databases therefore need not be identical.

- *Selective replication*

HVR can be configured to selectively replicate changes in a given table. Changes to rows that correspond to a definable criterion are replicated, whereas changes to other rows are ignored. This can be used, for example, to allow old rows to be purged from a source database that is used for production purposes, but to be kept in the target database that is used for archiving or reporting. It can also be used for horizontal partitioning. This means that different parts of a table are replicated into different directions. For instance in cases in which data from a central source database has to be replicated to multiple local target databases, a different subset of a table in the central database may be replicated to each corresponding table in the target databases.

- *Name and data conversion*

It is possible to configure replication between tables or columns that have different names between the source and the target databases. Moreover, instead of one-to-one copy of the column values HVR can be instructed to calculate new column values for the target database table from the source database table column values using an SQL expression, or to provide a default value. As a consequence, the source and target database tables are allowed to have different columns. Columns that are present in the source database table but not in the target database table may be ignored or can be used to provide input to calculate values for another column in the target database. Alternatively, columns that are present in the target database table but not in the source database table may be filled with a default value or a calculated value.

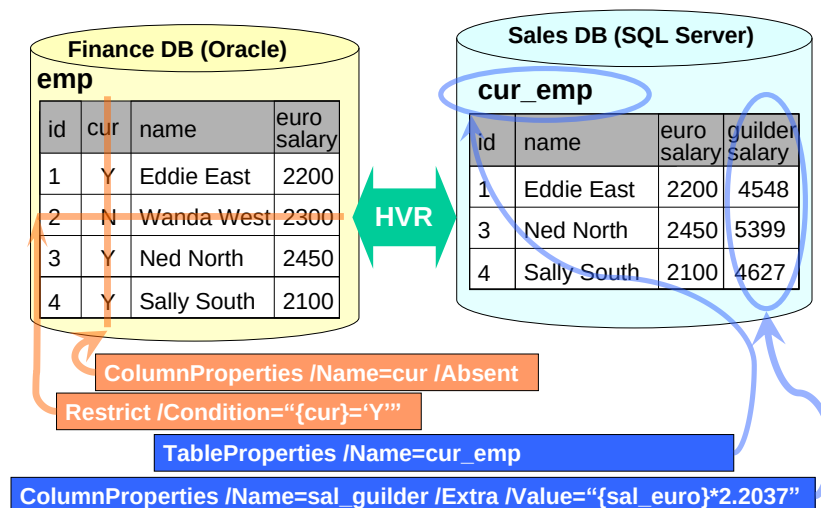


Figure 9 Name and data conversion

2.3 Database Compare and Refresh

The HVR Database Compare and Refresh functionality is a very elegant tool for bringing two databases into sync with each other. This can be used in itself for fast batch data loading, but it can also be used as a valuable complement to Continuous Database Replication. In the latter case Database Compare and Refresh can be applied to synchronize databases prior to enabling replication between them or to resynchronize replicated databases after a crash has occurred. The data transformation rules that were defined for the Continuous Database Replication can be reused for the Database Compare and Refresh.

Database Compare and Refresh shares many of the features and benefits of Continuous Data Replication. Specifically, it can be deployed in heterogeneous database environments and between databases with different schemas and it can handle large data volumes through its high performance and efficiency.

As the name already suggests, Database Compare and Refresh offers both the option to compare the contents of two databases and to synchronize (refresh) a target database with a source database. Database Compare can in itself be useful to monitor and verify the consistency of the databases. Both database Compare and Database Refresh can be done in two modes, a bulk mode and a row-wise mode:

- Bulk compare**
 In the case of bulk compare, HVR performs checksums on each of the corresponding tables in the compared databases and subsequently compares these checksums. The actual data does not travel over the network, so this method is very efficient for verifying that large databases are equal.
- Bulk refresh**
 When applying bulk refresh HVR reads the data from all tables in the source database. If the databases are on different servers, the data is compressed and transported over the network. As the final step, the data is loaded into the target database whereby the table indexes are reinitialized. Bulk refresh is the fastest option to perform a database copy.
- Row-wise compare**
 Row-wise compare means that HVR extracts the data from the source database, submits it in compressed format over the network (if required), and then compares the data table by table and row by row with the target database. For the detected differences it then formulates the minimal SQL script required to apply the inserts, updates or deletes to the target database to make it consistent with the source database.
- Row-wise refresh**
 Row-wise refresh consists of the same steps as row-wise compare, but the resulting SQL script is then applied directly to the target database to resynchronise it. Row-wise refresh is efficient in cases where the source and the target database are largely equal.

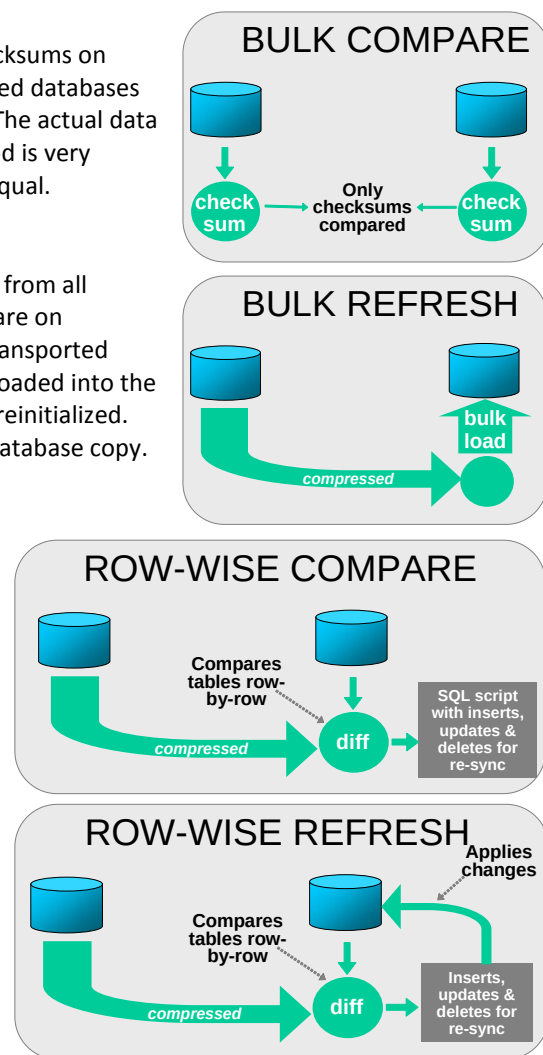


Figure 10 Compare and Refresh modes

All modes of Database Compare and Refresh are very flexible. They respect any transformation rules that may have been defined for a Continuous Data Base replication, such as selective replications and name and data conversions. Hence, the source and the target databases can have different schemas and contents.

Database Compare and Refresh is optimized for speed and flexibility. For instance, the data streams are parallelized. HVR orders the tables in groups of similar size by checking the DBMS catalogue information on the table sizes to ensure optimal load-balancing across the streams and processes. Moreover, the source database does not have to be taken off-line during a refresh action to avoid loss of changes. HVR will capture all changes applied to the source database during the refresh process and include them in the update of the target database. In case row-wise refresh is applied, it is even not necessary to take the *target* database off-line.

2.4 Managed File Transfer

In most organisations a large part of the exchange and distribution of information is realised by the copy and transfer of data files. This may result in countless numbers of unsecured and unmonitored point-to-point connections using various protocols, such as FTP or proprietary protocols. However, for business critical purposes a more infrastructural solution is required. Managed File Transfer allows for the centralized management and control of end-to-end file transfers within the enterprise. It also enables secure and auditable file transports and it can be used between disparate platforms, applications and people.

HVR provides Managed File Transfer functionality. This can be used as a tool in its own right, but also as part of the integrated HVR suite for enterprise data integration. Complex file transfer chains can be configured, scheduled and controlled from a central point on an enterprise-wide level. HVR supports 3 types of file transfer: file-to-file, database-to-file and file-to-database.

- *File-to-file transfer*

An HVR file-to-file transfer will copy the files from one file location (the source location) to one or more other file locations (the target locations). A file location is a directory or a tree of directories, which can either be accessed through the local file system (Unix, Linux or Windows) or through a network file protocol (FTP, FTPs, sFTP or WebDAV). Files can be copied or moved. In the latter case, the files on the source location are deleted after they have been copied to the target locations. The file contents are normally preserved, but it is possible to include file transformations in the copy process using external commands or definitions defined in an XSLT file. Furthermore, it is possible to copy files selectively from the source location by matching their names to a predefined pattern. This feature also enables the routing of files within the same source location to different target locations on the basis of their file names.

- *File-to-database*

In a file-to-database transfer data will be read from files in the source file location and replicated into one or more target databases. The source files are by default expected to be in a specific HVR XML format, which contains the table information required to determine to which tables and rows the changes should be written in the target database. It is also possible to use other input file formats by including an additional transformation step in the file capture. Support for CSV is already provided, but any format can be handled by providing an external command or an XSLT definition.

- *Database-to-file*

Alternatively, in a database-to-file transfer the data is read from a source database and copied into one or more files on the source file location. The resulting files are by default in the HVR XML format, so that the table information is preserved. However, CSV is also supported and other file formats can be obtained by including an additional transformation command or XSLT definition in the file output. As in the Continuous Database Replication between databases, it is possible to select specific tables and rows from the source database and convert names and column values.

It is no surprise that HVR Managed File Transfer is optimized for maximal performance, efficiency and scalability as it benefits from the same mechanisms for data compression, data encryption, network efficiency and queuing as Continuous Data Replication and Database Compare and Refresh. It therefore can handle multi-gigabyte files sent in a single task to or from 100 or more locations.

2.5 HVR Management and Operations

User Interface

All HVR tasks can be managed from a central intuitive and integrated Graphical User Interface (GUI). The GUI connects to the HVR hub and can be directly installed on the hub machine. Alternatively, it can be installed on the user's PC and connect to the hub machine over the network. Through the hub all HVR agents can be controlled and monitored.

The GUI can both be used to configure HVR tasks, such as Continuous Database Replication or Managed File Transfer schemes, and to schedule, execute and monitor these tasks. This is supported by the extensive built-in user manual.

Instead of using the GUI, most HVR actions can also be initiated from the command line and hence be included in scripts for automating operations.

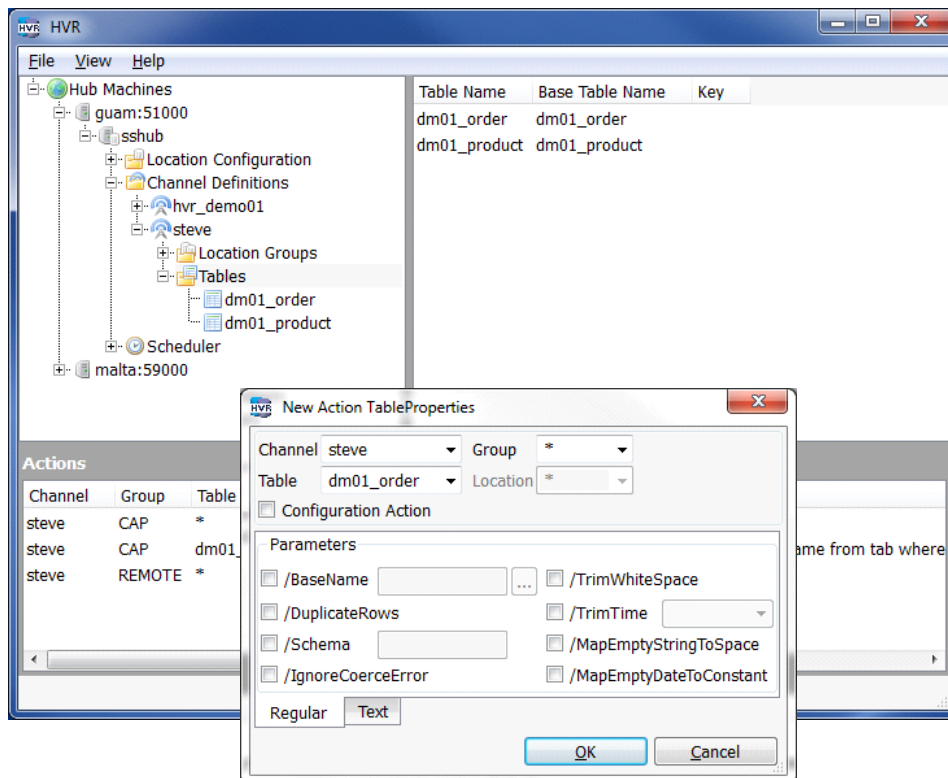


Figure 11 HVR GUI – Configuration Panel

HVR Channels

Setting up or modifying a HVR task consists of two parts: Location Configuration and Channel Definition.

- Location Configuration**
 The Location Configuration contains the parameters that describe the addresses and the access credentials of each database and file store ("location") that will be involved in the HVR task. Typical parameters are network node names, port numbers and login passwords. The type of storage (database or file store) and the kind of database (e.g. Oracle or Ingres) is also defined here. A logical name is assigned to each location.

- *Channel Definition*

In the Channel Definition the logical transformation rules for the HVR task are specified. The locations are referenced by their logical names assigned in the Location Configuration. The locations are combined into location groups which act as the source and target of the HVR operation. Furthermore, the tables can be selected which should be replicated. Finally, the actions for the channel are defined determining the type of operation (e.g. a database replication consisting of a capture and an integration action) and the options and parameters that should be applied (such as recalculating a column).

Once an HVR task has been configured it can be loaded to the hub machine and scheduled for execution.

The distinction between the Location Configuration and the Channel Definition parts of the configuration process allows for flexibility in terms of reuse and role separation. As the implementation details of the environment are covered by the Location Configuration and hidden from the Channel Definition, it is easy to change “physical” parameters within the environment, such as addresses or even replacing one type of DBMS by another, without having to change the “logical” parameters in the Channel Definition. Location Configuration could be done by the system administrators, without them having knowledge of the replication logic. On the other hand, Channel Definition can be done by the developers of replication schemes who do not need to be aware of the particular details of the deployment environment. Thus, a certain Channel Definition could first be tested within a testing environment and then be deployed to the production environment by only changing the Location Configuration.

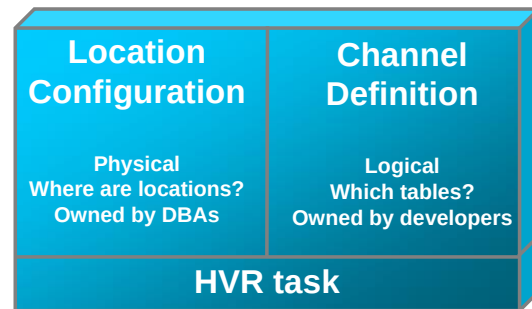


Figure 12 HVR configuration

Monitoring and Statistics

The monitor that forms part of the GUI provides status information and statistics on the progress of the various HVR tasks on all involved nodes in the distributed environment. HVR can also create alerts and interface to popular system management tools, such as HP Open View and Nagios.

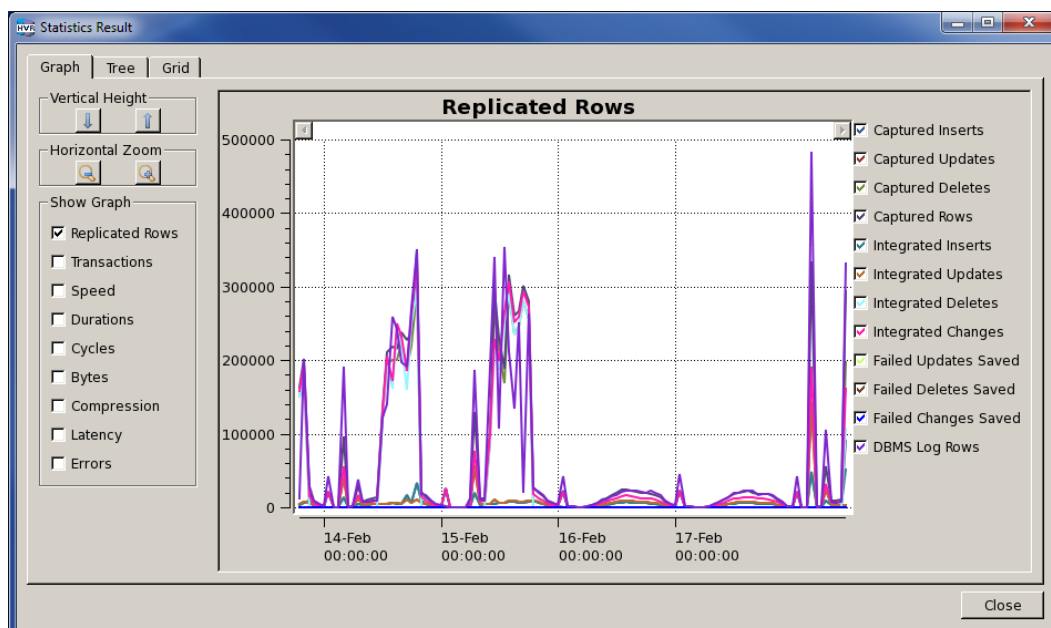


Figure 13 Monitoring

2.6 Platform Support

HVR has specifically been designed for data integration within large heterogeneous computing environments where divergent hardware platforms, database products and file systems coexist. Consequently HVR can be installed on a number of Operating Systems and interface with most of the mainstream DBMS products and file systems.

Supported Operating Systems

HVR can be installed on the following Operating Systems:

- Linux for x86 and x86-64
- Microsoft Windows for x86 and x86-64
- Solaris for Sparc and Intel
- HP-UX for Itanium
- IBM AIX

Supported Database Management Systems

HVR can interact with the following DBMS products:

- Oracle Database and Oracle Exadata
- Microsoft SQL Server and Microsoft SQL Azure
- Ingres
- Vectorwise
- IBM DB2 for Linux/Unix/Windows
- Teradata Database
- Salesforce.com

Supported File Locations

HVR can directly access files in directories on the local file system of the servers on which it is installed. The following file systems are supported:

- Unix and Linux file systems
- Windows file systems

Alternatively, HVR can access files on remote servers via the following network file protocols:

- WebDAV (e.g. to Microsoft Sharepoint)
- FTP and FTPs
- sFTP

3 HVR Usage Scenarios

The features and capabilities of HVR make it a very versatile toolset, that can be applied in a variety of usage scenarios within the enterprise. This has been proved extensively in a number of real customer cases.

3.1 Real-time Business Intelligence

Since many years Business Intelligence tools have been deployed to extract data from multiple production systems, and to subsequently combine and manipulate this data to generate information that can be used for reporting and analysis. The emphasis has thereby been on less time-critical strategic and tactical decision-support. In today's competitive environments, however, the need arises for using the combined data from multiple sources for much faster decision-making and operational control. Data sources are not only the internal production systems, but also include external sources and the amount of data grows ever larger. The resulting information needs to be almost or completely 100% up-to-date. Business Intelligence thus evolves into *Operational* Business Intelligence (OBI).

The traditional approach to Business Intelligence has been to follow a multi-step process. In the first step, called ETL (Extract, Transform and Load), data is extracted from the production systems or databases, converted and integrated, and loaded into a data warehouse. A data warehouse is essentially a large database with an optimized structure for analytic queries. In the next step so called data marts may be generated. These are databases with selected datasets for specific business areas, such as marketing. In the final step, either reports can be generated or interactive analyses can be done by querying the data marts or the data warehouse. Especially the ETL step can be a time consuming process, as it implies a lot of data processing. It is usually implemented as a batch process, running periodically (e.g. nightly), to reduce interference with the production systems.

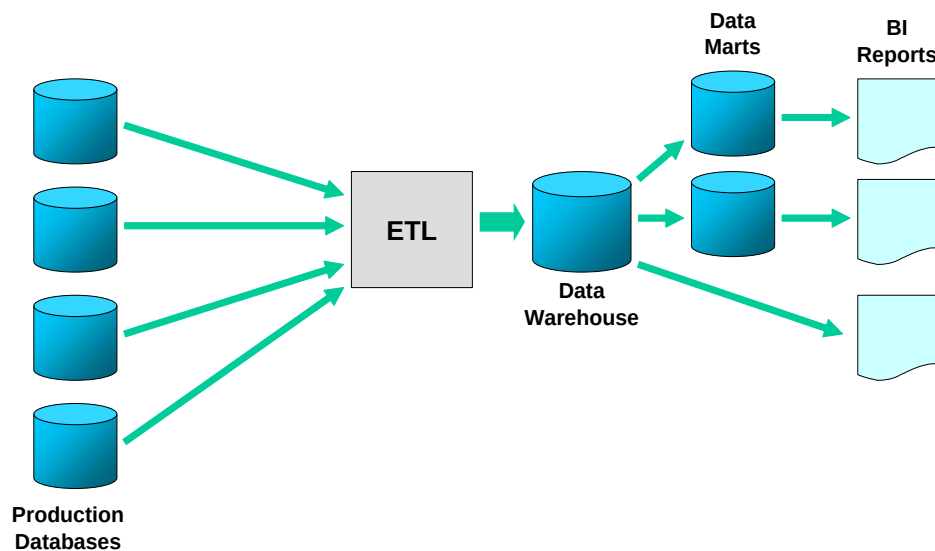


Figure 14 Traditional BI setup

It can be concluded that the traditional BI process is not well suited to support the zero-latency requirements of OBI due to the time-consuming ETL process. It is therefore necessary to include additional or alternative steps and components. Real-time capabilities can be obtained by introducing a "live" copy of selected primary production databases by using database replication and synchronization techniques. This live copy can be used as a source for real-time reporting and analytics. Obviously, this does not allow for the sophisticated data integration and conversion actions that can be implemented by contemporary ETL tools. ETL processing will therefore still be required for

advanced reporting and analysis. However, basic database transformations, merges or splits can be achieved when using appropriate replication tools and this could already cover most of the requirements for real-time reporting.

Consequently, a typical OBI setup will contain both a real-time replica from the production databases that serves as the source for OBI reporting, and a data warehouse that can serve as the source for more elaborate consolidated reports and for off-line analysis.

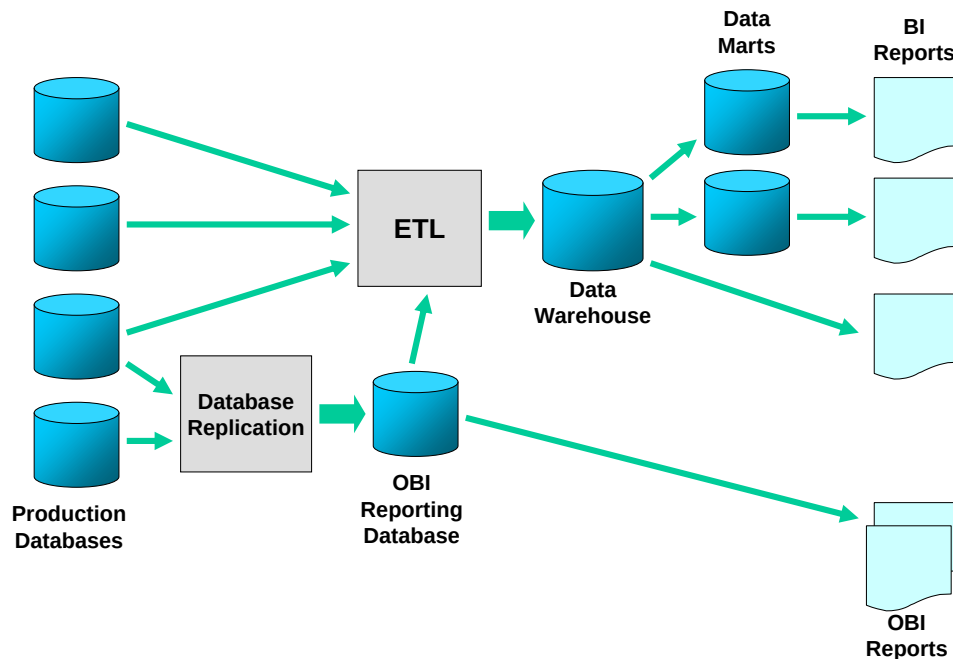


Figure 15 Extended BI setup for OBI reporting

HVR: the pre-eminent tool for Operational Business Intelligence¹

HVR has all the attributes and properties that are required to serve for database replication and synchronization within an OBI setup. The following ones can be highlighted:

- Data latency between the OBI reporting database and the source databases from which it is replicated can be kept at a minimum by HVR's mechanisms for fast and efficient network transport, including data compression and smart data packaging, and for efficient loading of data into the target database, such as coalescence of changes. Especially, when operating in log-based capture mode, whereby the changes are read from the log files of the source database, data latency can be less than 1 second.
- Interference with the production databases and servers is also kept at a minimum, avoiding any risks for performance degradations. When operating in log-based capture mode, HVR is totally unobtrusive to the source database, but even in trigger-based capture mode, whereby changes are recorded into and read from capture tables within the source database, overheads stay within limits of a few percent. Moreover, the HVR processing that takes place on the servers where the source databases reside is efficient in using system resources.
- HVR can work within heterogeneous environments, with various kinds and versions of database servers running on several platforms. This is very useful for OBI, as data will have to be extracted

¹ A much more extensive discussion on approaches to OBI in relation to HVR can be found in the white paper by Rick. F. van der Lans: "Empowering Operational Business Intelligence with Data Replication", which can be obtained from HVR Software bv.

from multiple applications that may use different database and server technologies.

- HVR is able to perform basic transformations when replicating tables in the various production databases to tables into the database that is used for OBI reporting. Transformations include selective replications of parts of tables, conversion of table and row names, and calculation and manipulation of column values. For instance, timestamps and so called soft deletes can be added to retain historical data in the OBI reporting database. Soft deletes are records that indicate that a particular record has been deleted from the source database. The timestamps and soft deletes are also very useful if the OBI reporting database is used as a source for subsequent ETL processing.
- The Database Compare and Refresh function of HVR allows to synchronize the production databases with the OBI reporting database as a prerequisite for database replication. It can also be used for recovery of a database that was taken out of operation.
- HVR is easy to configure and to operate through its user friendly centralized GUI.

3.2 Enterprise Application Integration

Enterprises usually employ a large set of applications to support their primary business operations and their back-office processes. Some applications may have been custom developed, some may be off-the-shelf software packages from various third-party vendors, while others may be subscriptions on Software-as-a-Service offerings. Legacy applications could also be part of the application portfolio, as well as newer applications complying with current web-based technologies and standards. The applications typically run on multiple server and middleware platforms, both within the enterprise or externally hosted.

Application Integration addresses the challenge to let these applications collaborate to deliver the combined functionality that is needed by the business. It covers the following integration objects:

- **Processes**
Business processes are usually supported by multiple applications. To automate the process chains the applications involved need to work together by exchanging transactions, status updates and documents. For instance, an order taken by a web shop application should be notified to a warehouse management application to trigger order picking and sending.
- **Data**
Business data can have multiple occurrences and formats in the various applications within the enterprise. For instance, customer information is used in the CRM application, but it needs also be available in the invoicing application. Preferably, the data within the different applications applying to the same business object (in this case a customer) should be kept consistent and complete by exchanging and converting the data in an automated way.
- **People**
Employees and customers need to have access to several different applications and other resources within the enterprise. Identity and access management of the users across all applications is therefore crucial for efficiency, usability and information security.

Application Integration styles

In any case Application Integration requires the applications in scope to exchange data to realise the required interworking. For this exchange of data different approaches can be followed, which can be grouped into two main categories or “integration styles”: Data-oriented Application Integration and Transaction-oriented Application Integration.

- **Data-oriented Application Integration**
Data-oriented Application Integration is based on the principle that the collaborating applications operate on a shared logical set of data. This does not necessary imply that they share the same physical data repository, but it does require that the data sets on which they operate have the same logical content. The application integration mechanisms and tools should therefore ensure that the data set instances on which the applications are operating are consistent, while taking care of any conversions required to comply with the local data formats supported by the applications or the infrastructure. Data-oriented Application Integration comes in two “sub styles”:
 - **File Transfer**
One application (the producer) writes the shared data into files, which are then transported to and read by the other application (the consumer). Especially in large environments involving many applications and complex file transfer patterns Managed File Transfer

capabilities will be required.

- *Shared database*

The applications operate on the same database or use replicated databases in which they store their data.

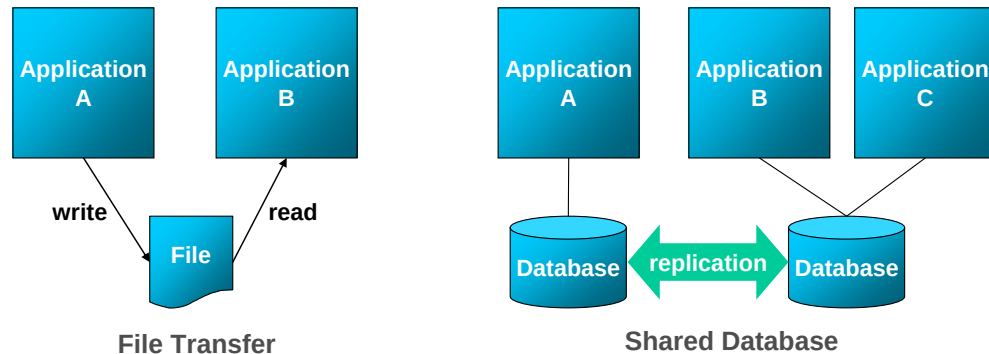


Figure 16 Data-oriented Application Integration

- **Transaction-oriented Application Integration**

Transaction-oriented Application Integration is based on the principle that one of the collaborating applications invokes a specific function or behaviour of the other application by sending it a call or message also containing the associated data. Again two “sub styles” can be distinguished:

- *Remote Procedure Invocation*

One application directly connects to the other application using one of its exposed interfaces to call a specific procedure or function

- *Messaging*

The applications connect to a common messaging queuing system (bus) to which they can submit messages or, alternatively, from which they can pick up messages, containing both transaction requests and the associated data.

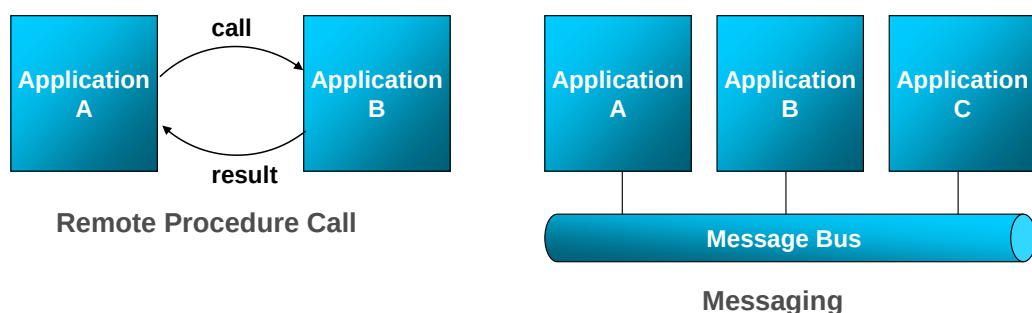


Figure 17 Transaction-oriented Application Integration

There is not one best application integration style, as they each have their specific advantages and disadvantages. In practice, they are often combined. Data-Oriented Application Integration puts few restrictions on the applications involved and it is relatively easy to accomplish and manage in heterogeneous environments. It can therefore be used in most cases. Transaction-oriented Application Integration on the other hand may offer a higher level of interworking functionality, but it is usually more complex to setup and maintain and the applications involved need to support the required interfaces.

HVR: the pre-eminent tool for Data-oriented Application Integration

HVR qualifies as the pre-eminent tool for Data-oriented Application Integration scenarios. It both supports integration through file transfer and shared database through its Managed File Transfer and Continuous Database Replication capabilities. Its specific properties make it ideally suited for large scale application landscapes:

- It can operate in heterogeneous environments involving different types of database servers and file systems.
- It can perform a whole range of data conversions needed to comply with the specific platform and application requirements.
- It can perform almost real-time database replications.
- It can be scaled out almost indefinitely.
- It can be operated from a single central point within the enterprise, while still maintaining high availability.
- HVR is easy to implement and to maintain.

3.3 Geographic Distributed Computing

Many businesses and organisations operate from multiple locations, varying from local branch offices within one country to the regional headquarters of a multinational enterprise. In support of these operations business data should be available at each individual location in a timely manner. On the other hand, data generated by the local operations usually needs to be consolidated into a central application or data store.

There are basically two approaches to achieve the local availability of business data:

1. **Centralized Data**

The data is kept at a central site, mostly a data centre, and is accessed from the local (or regional) systems or applications over a Wide Area Network.

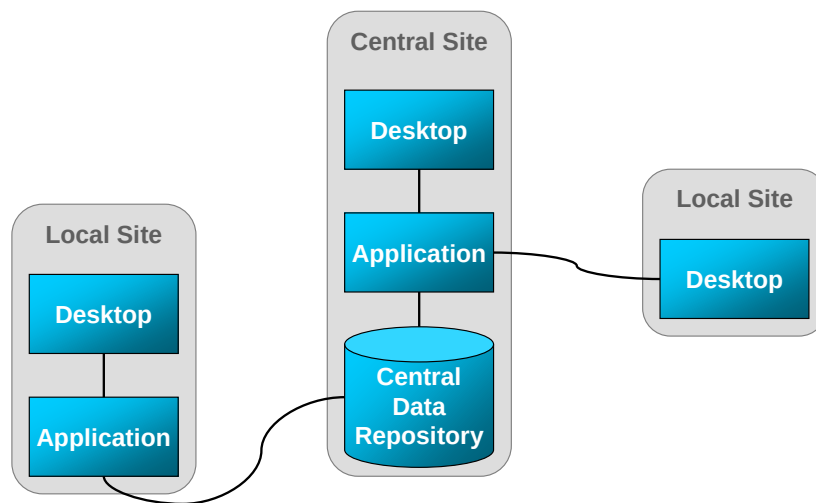


Figure 18 Centralized Data

2. **Geographic Distributed Data**

Each site features its own data repository, such as a database within a branch office or within a regional data centre. The local systems or applications operate on the associated local data repository. Data is shared between central and local data repositories through data replication and synchronisation over a Wide Area Network.

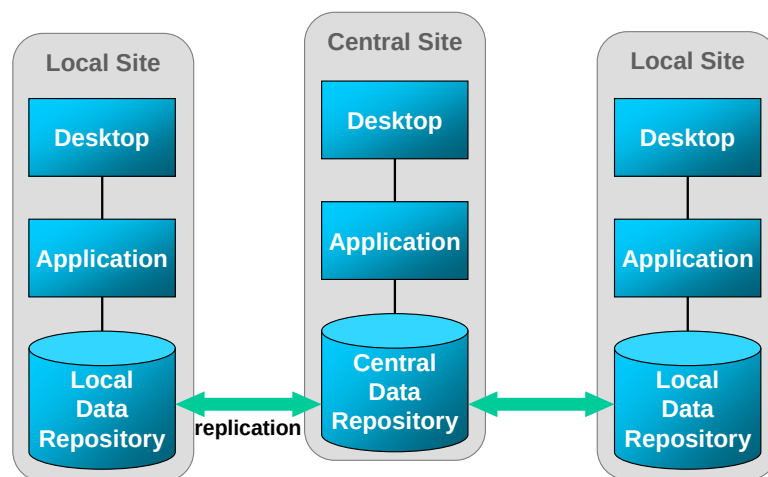


Figure 19 Distributed Data

HVR: the pre-eminent tool for Geographic Distributed Computing

HVR can be considered as a most adequate tool in support of a Distributed Data setup. It can synchronize and replicate databases and files between geographically dispersed sites thereby offering a range of useful features and options:

- HVR is extremely efficient over WAN due to its built-in data compression, smart data packaging, data management and coalescing of changes.
- SSL encryption can be enabled to secure data transport over non-trusted networks.
- HVR is very robust and reliable. It can detect and resolve database collisions and it can automatically deal with temporary disruptions in the network or data repositories. In any case, local disruptions will not hamper the replication and synchronisation to other sites.
- HVR is flexible, locations can be added or removed with a minimal configuration effort.
- Horizontal partitioning can be applied to selectively split a (central) database table for replication to different (local) sites or, reversely, to consolidate data base tables from various (local) sites into one (central) database table.
- HVR can operate between different brands or versions of database servers. It is therefore not necessary to run the same database server brand or version on each of the sites.
- HVR is able to handle up to hundreds of sites.
- HVR can be controlled, managed and monitored from a central point and GUI within the enterprise, while still maintaining high availability.

3.4 High Availability

In today's economy of internet-based services, more and more IT systems need to be available on a 24/7 basis. As a consequence, for most business improving the availability of their applications is a major area of interest and spending. High availability planning focuses on the following goals:

- *Minimizing performance downgrades and losses*
Reducing the frequency and duration of occurrences in which the applications are not available at all or their performance is below an acceptable level. Such occurrences may be caused by disruptions, ranging from minor incidents to major disasters, but also by scheduled downtime.
- *Minimizing data losses*
Reducing the effects of disruptions on data integrity or data losses.

Database Availability

As most critical enterprise applications rely on databases to store their data, the availability and the performance of these databases become a crucial part of the high availability strategy. A common approach is to replicate a critical database to a secondary database, to which a switch over can be done or to which traffic can be off-loaded. More specifically, the following use cases can be distinguished:

- *Incident resolving and disaster recovery: avoiding unplanned downtime*
Switch over to a secondary (standby) database replica can serve to resolve incidents that cause the primary (production) database to fail, but it can also be part of a disaster recovery scenario involving the complete fail-over of IT operations to an alternative data centre. In the latter case, the secondary database needs to be located at the other data centre, hence replication must be done over a WAN connection.
- *Zero-downtime maintenance: avoiding planned downtime*
Planned downtime occurs when a production database has to be taken off-line to perform scheduled operations, such as regular backups or cleanups, that can not be executed while the system processes are still running, users can connect to it, or changes or transactions may be applied to the data. Planned downtime can be avoided by switching over the operation from the primary (production) database to the secondary (standby) database replica. The maintenance can subsequently be applied to the primary database, after which the primary database is synchronized with the secondary database and operations are reverted back to the primary database.
- *Load balancing and off-loading: ensuring performance*
Whereas in the above cases the secondary database normally operates in a standby mode, it can also be used for production purposes. Thus traffic can be shared between the replicated databases, enhancing the overall performance. In that case the databases are included in a load balancing setup, in which they have equal roles and both can process read and write transactions. Alternatively, the secondary database could be configured as a slave processing read-only transactions, for instance for reporting or backup purposes.

HVR: the pre-eminent tool for Database Availability

Through its Continuous Database Replication and Database Compare and Refresh functionalities HVR is the tool of choice to replicate and synchronize production databases for all of the above Database Availability cases:

- HVR Continuous Database Replication capabilities for high efficiency and high speed, even over WAN connections, ensure a minimal loss of data in case of incidents or disasters. When the log-based capture option is applied data losses can be reduced to less than a second.
- HVR Database Compare and Refresh can be applied to easily (re)synchronize a database with the current production database, after it has been taken off-line for error recovery or maintenance.
- HVR can both be used for one-directional (master-slave) replications, in case the secondary database is purely used for standby or serves as a reporting database, but it can also be used for bi-directional replications, in case both databases serve for load-balanced production. In the latter case data collisions will be detected and recovered.

3.5 Migrations

Major changes and implementation projects are a fact of life in any large computing environment. They are necessary to fulfil the ever changing requirements from end users or the business, to benefit from technical developments and opportunities, and to upgrade or replace hardware and software components that have become outdated. In most cases such migrations can not be carried out in isolation, but they have to be applied to systems supporting primary operations that may not be interrupted. This puts severe requirements on database availability during upgrades and replacements and on the coexistence of legacy systems and new systems in the course of a migration project.

- *Database availability during upgrades and replacements*
Zero-downtime of the production databases during software upgrades or hardware replacements can be achieved in the same way as for zero-downtime maintenance. Prior to the actual migration, production is switched over from the primary database to an up-to-date replica. Afterwards the migrated (or replaced) primary database is synchronized with the replica running production. Finally, production is returned to the primary database.
- *Coexistence of legacy systems and new systems*
Especially in large and long lasting migrations involving large databases or file stores it is not always possible to carry out the transfer of data from the old (legacy) systems to the new systems in one go. In those cases a phased approach has to be adopted in which the data is set over step by step during a longer period. In this period the old and the new systems have to cooperate by exchanging and sharing data (databases and files) to ensure continuity of service.

HVR: the pre-eminent tool for Migrations

The benefits that HVR provides for Database Availability and Data-oriented Application Integration scenarios combine to make it also extremely useful to support Migrations. HVR can both be used to cater for zero-downtime of production databases during upgrades and to facilitate data sharing between legacy systems and new systems by replicating databases or managed file transfers.

3.6 Heterogeneous Cloud Computing

In recent years Cloud Computing has become a major trend in the IT market. The proliferation of the internet and associated web standards and computing architectures supporting infinite scalability and multi-tenancy are the main enablers to provide computing resources as a standard service delivered over a network (usually the internet) on a pay-per-use basis. For the users cloud services promise reduced and flexible costs, shorter times-to-market and more business focus. Most businesses are therefore considering or already implementing cloud services.

Cloud computing services come in a large variety of appearances. These are usually grouped into one of the following categories:

- *Infrastructure-as-a-Service (IaaS)*
Cloud services providing virtual machines with their associated operating system and logical file stores, and network connectivity. A well-known example is Amazon Elastic Compute Cloud (EC2).
- *Platform-as-a-Service (PaaS)*
Cloud services providing a runtime environment including middleware components, such as databases, portals, message queues, service buses, web servers, web containers and software libraries, on which applications can be deployed and executed. For instance, Microsoft Windows Azure is usually classified as a PaaS.
- *Software-as-a-Service (SaaS)*
Cloud services providing full application functionality, such as Sales Cloud from Salesforce.com for enterprise customer relationship management.

Most enterprises have extensive application portfolios that are hosted on a dedicated infrastructure, which can be internal or delivered and managed by an infrastructure provider. Adoption of cloud services usually implies that they add or replace applications or infrastructure components by subscribing to various cloud services in an incremental fashion. Their IT landscape therefore transforms into a combination of dedicated infrastructure components and applications and a variety of cloud solutions. This situation can be referred to as “Heterogeneous Cloud Computing”.

HVR: the pre-eminent tool for Heterogeneous Cloud Computing

In such heterogeneous cloud computing environments, all of the above HVR usage scenarios for data replication and synchronization in support of Operational BI, Application Integration, Distributed Computing and High Available may occur. Moreover, they have to be implemented between and over the different dedicated systems and the cloud based platforms.

Already through its standard capabilities for operation in heterogeneous environments over wide area networks, HVR can be deployed in many situations where database replication and synchronization or file exchange with or between cloud environments have to be accommodated. For HVR it makes no difference if a replication source or target database is hosted on an IaaS platform, as long as both the database and the hosting platform comply with HVR’s platform requirements, such as an Oracle Database that is hosted on Amazon EC2.

Additionally, HVR is being extended to interoperate with specific cloud databases that are included in PaaS or SaaS platforms. In this manner HVR is already capable of replicating and synchronizing with the databases within the cloud platform of Salesforce.com, that is used for their services Sales Cloud, Force.com and Database.com. HVR has also been adapted to interoperate with the MS SQL database instances that are provided within Microsoft Windows Azure

4 Conclusion and Outlook

For the majority of companies business data is one of their most valuable assets. The ability to manage and control the sharing and distribution of data to get the right information at the right place and on the right time is therefore a prime strategic competence, also considering that the amount of data is growing exponentially and real-time operation becomes the standard.

HVR provides a range of capabilities that allow businesses to distribute, replicate and synchronize their databases and files, thereby addressing a variety of needs and usage scenarios and covering both structured and unstructured data. Its properties make it the tool of choice to operate in large scale setups:

- HVR can operate within heterogeneous and distributed environments, including cloud platforms.
- HVR is optimized for large data volumes, low latencies and robustness.
- HVR is extremely scalable.
- HVR is easy to manage through its centralized GUI.

HVR can typically be applied as a solution for Operational Business Intelligence, Data-oriented Application Integration, geographically Distributed Data, Database Availability, Migrations and Heterogeneous Cloud Computing, but thanks to its generic capabilities various other usage scenarios can also be envisaged.

Since its origin as a solution for distributed computing and high availability, HVR has been continuously developed into the state-of-the-art data integration suite that it is today with enterprise class capabilities for application and cloud integration, operational data warehousing and Managed File Transfer. Current developments are targeted to extend the range of databases and platforms with which HVR can work, including No-SQL databases and additional cloud platforms.

5 Further information

For further information please contact:

Email: info@hvr-software.com

Internet: www.hvr-software.com

Telephone: +31 20 820 1570

Mail: HVR Software bv
Daalwijkdreef 47
1103 AD Amsterdam
The Netherlands

Copyright © 2013 HVR Software. All rights reserved. Trademarks referenced in this document are the sole property of their respective owners.